

Grade the Floor: A Read-Time Energy-Performance Rating for a Two-Floor Smart Building Monitor

Venkata Sai Mothukuri

Student ID: X25181238

MSc in Cloud Computing

Fog and Edge Computing (H9FECC)

National College of Ireland

x25181238@student.ncirl.ie

Abstract—A building manager needs two different readings of a floor, and a monitor that offers only one of them is half a tool. The first is an exception: a specific thing that has gone wrong and needs a response now, a plant drawing peak load, stale air, a room too hot or too cold, a tap left running. The second is a rating: an at-a-glance sense of how efficiently the floor is running overall, the kind of judgement a building energy certificate rolls into a single letter. This project provides both from the same stream and keeps them distinct. Ten sensor processes drive five signals, energy consumption, carbon dioxide, occupancy, air-conditioning temperature, and water use, across two floors; a fog node on the edge host windows each floor-and-signal stream, reduces it to a summary, and raises the hard exception alarms locally. Behind the edge, a serverless backend on Amazon SQS, AWS Lambda, and Amazon DynamoDB persists every window, and a dashboard served through Amazon API Gateway does the second job: on every read it scores each floor's energy use and air quality onto a common zero-to-one-hundred scale, averages them, and maps the result onto an A-to-F letter grade, the way a building carries an energy-performance rating. The grade is never stored; it is derived fresh on read. The system was provisioned to a real Amazon Web Services account rather than a local emulator; 133 automated tests pass, and verification against the live endpoint confirmed both floors streaming all five signals, each carrying a live efficiency grade, and a healthy pipeline, with the frontend delivered from Amazon S3.

Keywords—smart buildings, building energy management, energy efficiency, indoor air quality, fog computing, edge computing, Internet of Things, serverless computing, Amazon SQS, AWS Lambda, Amazon DynamoDB

I. INTRODUCTION

A commercial building spends most of its running cost on energy, and the floors that waste it rarely announce themselves with a fault. A plant can run inefficiently for weeks inside every safe limit, and stale air or a slow overuse of energy is a matter of degree rather than a discrete failure. Instrumenting a building to read those degrees continuously is the premise of smart building management, where dense sensing of energy, air, and occupancy is turned into control and reporting [1], [2]. The design question is where those readings become a decision.

The Internet of Things makes it cheap to instrument every floor and plant room [3], but streaming every raw sample from every meter to a distant cloud region to be judged there is wasteful in two directions at once. It spends bandwidth

continuously on readings that, taken singly, rarely change anything, and it defers the one moment that matters, a signal crossing into an exception range, until after a network round trip. Fog computing was proposed to close that gap by placing a compute tier between the sensors and the cloud, near enough to the sources to summarise, filter, and evaluate rules before anything travels [4], [5], and it is one expression of the broader movement of computation back toward the network edge [6], [7]. Behind that edge tier, the cloud supplies the elastic, pay-as-used model that removes the need to size a server for a peak that sits idle most of the day [8].

This project builds that two-tier design for a scaled but faithful slice of a monitored building: two floors, designated floor-1 and floor-2, each carrying the same five signals, run as ten independent sensor processes. What gives the design its own character is the separation of the two questions a manager actually asks. One is an exception question, answered at the edge, where the fog node raises a hard alarm the moment a signal crosses a safety or comfort limit. The other is a performance question, answered at read time in the dashboard, where each floor is rolled up into a single A-to-F efficiency grade in the manner of a building energy certificate. The two are deliberately different in kind: an exception is a discrete event that demands a response, while a grade is a smooth rating that summarises how well the floor is running, and a floor can carry a perfectly good grade while still tripping one exception, or grade poorly while tripping none.

The work pursues four objectives, each demonstrated on a running system rather than only described. The first is a sensor and fog layer that genuinely windows and evaluates five heterogeneous signals at sampling and dispatch rates that are configurable rather than fixed in code. The second is a backend that scales through managed, elastic AWS primitives instead of a single always-on server. The third is a dashboard that turns stored windows into an operational reading, in particular the graded A-to-F efficiency rating per floor. The fourth, and the one that separates a working system from a plausible one, is deployment onto a real public-cloud account and verification against the live endpoint. Scope is held to these layers at a two-floor scale rather than a whole building's worth of floors. The remainder of this paper is organised as follows. Section II presents the architecture and the reasoning behind each choice,

including the alternatives weighed and the security posture. Section III covers the implementation, the automated tests, continuous integration, the real deployment, and the evidence gathered from the running system. Section IV closes with a critical assessment of the trade-offs, the limits, and where the work could go next.

II. SYSTEM ARCHITECTURE AND DESIGN

Fig. 1 shows the deployed system end to end: ten sensor processes and the fog node sharing one edge host, a managed queue, two independently deployed functions, a key-value store, and a browser-facing bucket reached through a managed gateway.

A. Sensor and Fog Layer

Each of the ten sensor processes represents one meter on one floor. On every sampling tick a process advances a bounded random walk: it nudges its previous value by a step drawn from a small symmetric range and clamps the result into a physically plausible band for that signal, so consecutive readings stay close together like a real meter rather than independent noise. The step sizes are set per signal to match how the real quantity moves, so a carbon-dioxide level that rises and falls as a room fills drifts faster between ticks than an air-conditioning temperature that changes slowly. Each process runs sampling and dispatch as two independent self-rearming timer chains, each of which schedules only its own next tick when its body finishes, so tuning one interval never skews the other's cadence and a slow dispatch never delays the next sample. A short lock guards the small local buffer: the sampler appends under it, and the dispatcher swaps the whole buffer out under it for an empty one before shipping, so a reading sampled mid-dispatch simply lands in the fresh buffer, and on a failed send the batch is put back in front of whatever accumulated so nothing is lost or reordered. The sampling interval defaults to two seconds and the dispatch interval to ten, and keeping them separate lets a fast-moving quantity be sampled often while still being shipped in economical batches. Each dispatch is a compact JSON object naming the signal, the floor, and the unit and carrying the batch of values gathered since the last dispatch, which at the default rates is roughly five readings amounting to a few hundred bytes on the wire, well under a kilobyte, posted over HTTP to the fog node's ingest endpoint.

The fog node is a Python service built directly on the standard library's threading HTTP server, running as its own container on the edge host distinct from the sensor processes and the cloud functions, with hand-dispatched routing and no web framework. Every ingest request is validated before anything is buffered, so a malformed or incomplete batch is rejected with a clear error rather than corrupting a window. The buffering is where the concurrency is handled with care. Ingest requests are served on many threads at once, so rather than let each request thread reach into the shared window buffers and take a lock to extend its group, the request thread does the cheapest possible thing: it drops the batch onto a thread-safe inbox queue and returns. A single dedicated consumer thread

blocks on that inbox and is the only thread that ever touches the window buffers, folding each arriving batch into them one at a time. The buffer bookkeeping is therefore serialised onto one owner rather than contended across the request pool, and the request path never blocks on it. When the window boundary arrives on a fixed timer, that same buffer state is taken atomically and reset for the next window, and only the non-empty floor-and-signal groups are carried forward.

When a window closes, each non-empty group is reduced to a summary carrying the count, minimum, maximum, average, and latest value, and that summary, not any raw reading, is evaluated against the exception rules. Each rule is a frozen, self-validating descriptor that names the signal it applies to, the summary field it reads, a comparison, and a limit, and refuses at construction to name a field or operator it does not understand, so a malformed rule fails loudly at start-up rather than silently never firing. An average energy consumption above fifty-five kilowatts raises a peak-load alarm, an average carbon-dioxide level above one thousand parts per million raises a poor-air-quality alarm, an average water use above twenty litres a minute raises a leak-suspected alarm, and air-conditioning temperature carries two rules rather than one, because comfort is breached from both sides: an average above twenty-six degrees is too hot and an average below eighteen is too cold. Occupancy deliberately carries no rule; it is windowed and shown as the context that explains the rest, but a busy floor is not itself an exception. The fog node also publishes this rule set, as a descriptive table rebuilt fresh on each request so it can never drift from what is actually enforced, on a read-only endpoint the dashboard forwards.

The fog node's output interface is a real managed queue rather than a direct write to storage. On each window boundary it batches the closed summaries and sends them to Amazon SQS in grouped calls, each carrying up to the service's per-call limit of ten summaries, so a whole flush costs a number of send operations proportional to the summaries closing divided by ten rather than one send per summary. The queue's address is resolved once and then memoised, but a failed resolution is never cached, so a fog node that starts before its queue exists keeps retrying until the queue appears and then holds the resolved address for the rest of its life, and a failed flush is logged while the node keeps running rather than crashing.

B. Backend Layer

The cloud tier follows a single ingest path with a branch for the operator interface. A queue receives the fog node's summaries, one function drains the queue and writes to storage, and a second, separately deployed function reads that storage back to answer the dashboard. Splitting the write path from the read path is deliberate: ingestion and querying then scale and fail independently, so a surge of windows cannot slow the dashboard and a burst of operator activity cannot back up ingestion.

Queue instead of a direct call. The fog node could invoke the storage function directly and synchronously, dropping the queue and saving a hop. That option was declined. A

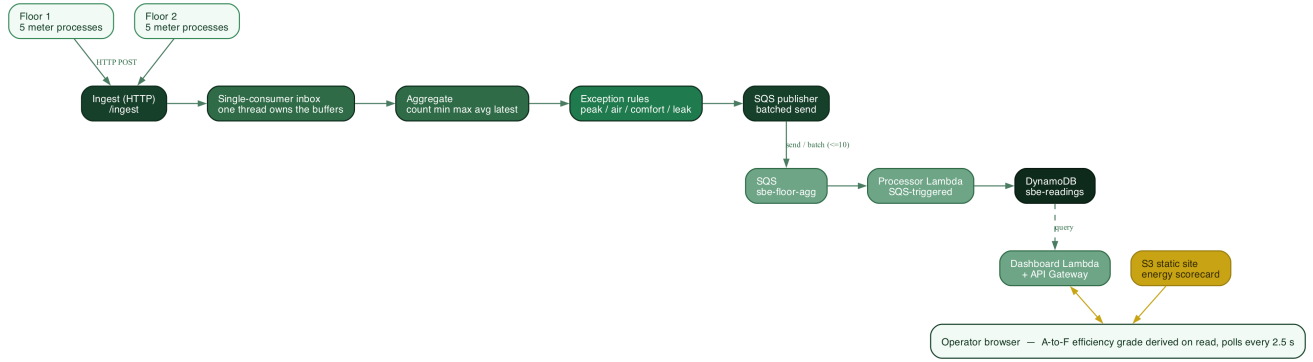


Fig. 1. Deployment topology. Ten sensor processes, two floors of five signals each, and the fog node share one Amazon EC2 edge host. The ingest path runs from the fog node through Amazon SQS and the processor function into the Amazon DynamoDB table; the serving path runs from the dashboard function’s read of the stored windows out through Amazon API Gateway to the browser, with static assets served from Amazon S3. At the edge, one dedicated consumer thread owns the window buffers exclusively, so ingest requests only enqueue. The A-to-F efficiency grade is derived at read time in the browser-facing function, not stored. The dashed edge is the dashboard function’s query back over the stored windows.

synchronous call binds the edge to the availability and latency of the cloud function: throttle it, leave it cold, or take it down briefly, and the fog node stalls or sheds data as the back-pressure travels back toward the sensors. A queue severs that coupling, absorbs bursts, retries on the consumer’s behalf, and lets the fog node treat a summary as delivered the moment it is enqueued [9]. On a ten-second cadence the small added latency is immaterial, while the decoupling and durability are exactly what an intermittent edge link needs.

Event-driven functions instead of a standing server. Both the ingestion and the interface tiers run as event-driven functions. This fits work that arrives in bursts and, for the interface, only intermittently: billing is per request rather than for idle capacity, and the platform adds and removes instances as demand shifts, with nothing reserved in advance [8]. The recognised cost is the cold start after a period of idleness [10]; its effect here is small, because steady queue traffic keeps the ingestion function warm and the dashboard’s own polling keeps the interface function warm. The processor function is triggered by the queue, holds no state between invocations, and is billed only for the work it does.

Key-value store instead of a relational database. A relational engine would offer joins and open-ended queries, neither of which this workload uses. Its access pattern is narrow and known in advance: append summaries in time order, then fetch the most recent windows for a given signal. Amazon DynamoDB, an on-demand key-value store built for that mix of heavy writes and key-based reads, holds its performance as the table grows and spares the operational overhead of running a relational server, the balance the original Dynamo work set out and the managed service now carries [11], [12]. The key design follows straight from the query. Each stored window is keyed on its signal as the partition and on a sort value that leads with the window’s closing timestamp and then the floor identifier, so windows for one signal are already ordered chronologically inside a single partition and the newest are read with one bounded, descending, single-partition query rather than a scan; the floor in the sort tail keeps the two

floors’ windows for the same signal from colliding.

C. Serving Layer, the Efficiency Grade, and Security

The dashboard reaches the read function through a managed gateway that publishes a public REST endpoint. Rather than write a second copy of the routing for the cloud, the read function drives the same request handler the standalone server uses, feeding each gateway request through an in-memory socket and reading the response back out, so one body of routing and query code runs both locally and in the cloud. Object storage holds the static frontend, one page, a stylesheet, the page script, and a charting library kept in the repository rather than pulled from a content-delivery network. The function exposes a small, read-only surface: a readings endpoint returning a recent series for one signal, a floors endpoint giving each floor’s latest window per signal together with its efficiency grade, a thresholds endpoint that forwards the fog node’s published rules, a backend-statistics endpoint, and a health endpoint assembled from four live sources, the store for the freshest window, the queue, the processor function’s deployment state, and the fog node’s reachability, returning a pipeline flag that is true only when a window actually reached the store recently.

The efficiency grade is the piece of interpretation the exception alarms do not provide, and it is computed here, on every read, rather than stored. For each floor the dashboard takes the two signals that most define energy performance, the energy draw and the air quality, and scores each on a common zero-to-one-hundred scale: a reading at or below an efficient threshold scores one hundred, one at or above a poor threshold scores zero, and anything between falls linearly, so the score expresses how far into the efficient-to-wasteful range the floor currently sits. The two component scores are averaged into one number and that number is mapped onto an A-to-F letter, exactly the shape of a building energy-performance certificate: a floor running lean on both counts earns an A, one drifting toward heavy use or stale air slides down the letters. Deriving the grade at read time

rather than storing it keeps the stored windows the single source of truth, lets the thresholds be retuned without rewriting anything already persisted, and yields a graceful pending state for a floor whose telemetry has not arrived yet rather than a wrong grade. On the page, each floor is a card showing its grade badge and score alongside its five raw readings, with an energy-consumption trend across both floors and any firing exception named as a tag; a fixed browser-side timer re-fetches the floors view, health, and statistics every 2.5 seconds and repaints from whatever the responses hold, so the view stays current without a manual reload and without any server-push channel. The page learns which gateway origin to call from a single placeholder written into it and substituted with the real endpoint when the assets are uploaded; run locally, where one process serves both the page and the interface, the placeholder resolves to empty and requests fall back to the same origin. The security posture is modest and deliberate for a scaled demonstration: the fog node's health and threshold endpoints answer without a credential check because they expose only aggregate window state and configured limits, never a live per-sensor reading, while the cloud tier relies on the managed services' own access control and on scoping each function to the queue, table, and gateway it needs.

III. IMPLEMENTATION

The sensor processes, the fog node, and both cloud functions are written in Python. The fog node and the dashboard's local server are built directly on the standard library's threading HTTP server with hand-dispatched routing, so neither carries a web framework; the AWS SDK for Python carries the queue, table, and function calls, and the frontend renders its trend with a locally vendored charting library so the page has no external dependency at load time. The pieces are kept as small single-purpose modules, the single-consumer ingest separated from the aggregation, the exception rules separated from the publisher, and the efficiency grade separated from the storage access, so each is unit-testable on its own without standing up a server or reaching into a live service. The edge tier is orchestrated with Docker Compose, the cloud tier is provisioned with Terraform, and continuous integration runs on GitHub Actions.

A. Testing and Continuous Integration

The system carries an automated suite of 133 tests spread across every component: the bounded-walk sensor and its retain-on-failure dispatch, the ingest validation, the single-consumer inbox and the atomic window drain, the window aggregation arithmetic, the self-validating exception rules, the batched publisher, the processor's transform into a stored item, and, on the dashboard side, the efficiency score and its letter-grade mapping, the per-floor assembly, the health checks, and the thresholds proxy. They run without any cloud connection by exercising the units against hand-written fake AWS clients, so each test can assert on the exact shape of the request a given call would issue rather than depending on a live service. The rule tests are pointed deliberately at the boundaries, checking

a value one step inside a threshold and one step outside it, and the two-sided comfort rule is tested from both directions so that too hot and too cold are each proven to fire; the grade tests check the score at the efficient and poor endpoints and at each letter boundary, so an A that should be a B is caught. The publisher tests confirm that a flush is split into batches no larger than the queue's per-call limit, the processor tests check the composite sort value that orders windows within a partition, and the fog ingest path is proven with a real HTTP request against a running local server rather than only a unit test of the validator in isolation, since that request path is what the deployment actually exercises. The suite runs on every push through GitHub Actions, so a change that breaks a threshold, a batch size, a query shape, or the grade mapping fails in continuous integration rather than after deployment.

Testing against fakes rather than a live queue was a deliberate choice with a cost worth acknowledging: a fake asserts on the request a call would make, not on how the managed service behaves under load, so the batched-send limit and the queue's own retry semantics were reasoned about from the service documentation [13] and then confirmed against the live deployment rather than in unit tests.

B. Deployment and Live Verification

The cloud tier was provisioned to a real AWS account in a single Terraform apply that created twenty-four resources, with the account confirmed before anything was applied and the working state isolated so the apply could add only its own resources and never disturb anything already present. Describing the whole cloud tier as code, rather than clicking it together once, means the same definition builds the queue, both functions with their triggers and permissions [14], the table, the gateway with its single catch-all route [15], and the two buckets in one reproducible step, and tears them down as cleanly. The edge host runs the fog node and the ten sensor containers under Docker Compose; all the deployed resources carry a common prefix so the deployment is easy to identify and to remove as a unit, and a fixed public address in front of the edge host lets the read function reach the fog node's health and threshold endpoints at the same location across restarts.

Verification was carried out against the live deployment rather than a local emulator, and Fig. 2 is the deployed dashboard during that check. Within about a minute of start-up the pipeline reported healthy end to end: the fog node, the queue, the processor function, and the freshest-window recency all read healthy, and the newest window was a few seconds old on each poll. Both floors rendered all five signals with their current value, and each floor carried a live efficiency grade computed from its energy and air readings, so both sat in the A band while the numeric score beneath the letter still separated a floor running very lean from one just beginning to drift toward heavier use. The energy-consumption trend showed the two floors moving independently across the window history, the backend-statistics endpoint showed the stored record count climbing across successive polls rather than a static number, and reaching the dashboard from a browser also exercised the

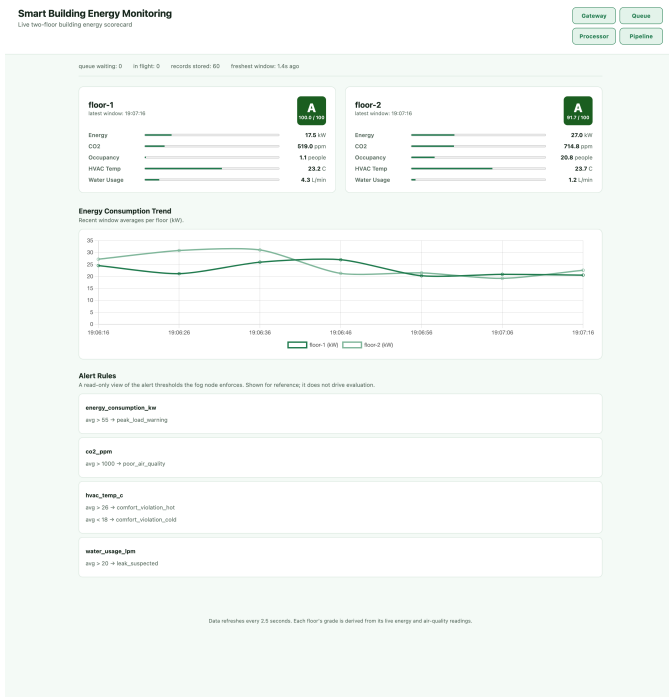


Fig. 2. The deployed dashboard reading from the live endpoint. Each floor shows its A-to-F efficiency grade badge and score alongside its five raw readings; an energy-consumption trend spans both floors; the header strip shows the four pipeline indicators; and the reference panel lists the fog node's exception rules, including the two-sided comfort band.

cross-origin path from the storage bucket to the gateway, which returned data cleanly.

IV. CONCLUSION

This project set out to build and, above all, to deploy a fog-to-cloud pipeline that turns five signals across two building floors into a live operational reading, and it met that goal on a running system rather than on paper. The edge tier does genuine work before anything travels: it windows and reduces each floor-and-signal stream and raises the exception alarms locally, so only compact summaries cross the network and an exception is named in the window it appears. The cloud tier scales through decoupling and elasticity rather than a pre-sized server, using a queue to absorb bursts and separate the write and read paths, event-driven functions billed only for the work they do, and an on-demand key-value store whose key design turns the dashboard's one query into a single-partition read. The whole system was provisioned to a real account by one infrastructure-as-code apply and verified end to end against the live endpoint, with 133 automated tests guarding its behaviour and continuous integration running them on every change.

The choice that shaped the system most was giving a manager a rating as well as an alarm. An exception alarm is precise but narrow: it says one thing is wrong and nothing about how the floor is doing overall. Rolling the two efficiency-defining signals into a single A-to-F grade, on the model of a building energy certificate, adds the missing dimension, a smooth summary of how leanly the floor is running, without

pretending to be an alarm. Keeping the two apart, the alarm raised at the edge and the grade derived at read time, meant each could use its own thresholds for its own purpose, and computing the grade on read rather than storing it kept the stored windows the single source of truth so the grading can be retuned without rewriting a single saved record.

The work has clear limits. It runs at a two-floor scale with simulated sensors, its exception limits and grade thresholds are fixed in code rather than learned from each floor's own baseline or adjustable at runtime, and its security posture is intentionally modest for a demonstration. Natural next steps are to widen it to a real building of instrumented floors and real meter feeds, to move the fixed thresholds to a managed configuration a facilities team could tune per floor without a redeploy, to weight the grade by occupancy so an empty floor and a full one are judged fairly, to add authentication and a custom domain in front of the public endpoint, and to retain longer history so a floor's daily and seasonal efficiency can be trended rather than only graded on the latest window. None of these changes the shape of the pipeline, which is the point: the edge-aggregation and cloud-serving design carries from two floors to many without rethinking the tiers.

REFERENCES

- [1] D. Minoli, K. Sohraby, and B. Occhiogrosso, "IoT Considerations, Requirements, and Architectures for Smart Buildings—Energy Optimization and Next-Generation Building Management Systems," *IEEE Internet of Things Journal*, vol. 4, no. 1, pp. 269–283, 2017.
- [2] Y. Agarwal, B. Balaji, R. Gupta, J. Lyles, M. Wei, and T. Weng, "Occupancy-Driven Energy Management for Smart Building Automation," in *Proc. ACM Workshop on Embedded Sensing Systems for Energy-Efficiency in Building (BuildSys)*, 2010, pp. 1–6.
- [3] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A Survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.
- [4] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog Computing and Its Role in the Internet of Things," in *Proc. MCC Workshop on Mobile Cloud Computing*, 2012, pp. 13–16.
- [5] A. Yousefpour et al., "All One Needs to Know about Fog Computing and Related Edge Computing Paradigms: A Complete Survey," *Journal of Systems Architecture*, vol. 98, pp. 289–330, 2019.
- [6] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [7] M. Satyanarayanan, "The Emergence of Edge Computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [8] M. Armbrust et al., "A View of Cloud Computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [9] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston, MA: Addison-Wesley, 2003.
- [10] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking Behind the Curtains of Serverless Platforms," in *USENIX Annual Technical Conference*, 2018, pp. 133–146.
- [11] G. DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store," in *Proc. ACM SIGOPS Symp. Operating Systems Principles*, 2007, pp. 205–220.
- [12] M. Elhemali et al., "Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service," in *USENIX Annual Technical Conference*, 2022, pp. 1037–1048.
- [13] Amazon Web Services, "Amazon Simple Queue Service Developer Guide," 2024. [Online]. Available: <https://docs.aws.amazon.com/sqs/>
- [14] Amazon Web Services, "AWS Lambda Developer Guide," 2024. [Online]. Available: <https://docs.aws.amazon.com/lambda/>
- [15] Amazon Web Services, "Amazon API Gateway Developer Guide," 2024. [Online]. Available: <https://docs.aws.amazon.com/apigateway/>

[16] IEEE, “Manuscript Templates for Conference Proceedings,” 2024. [Online]. Available: <https://www.ieee.org/conferences/publishing/templates.html>

The source code and deployment configuration are available at [GitHub repository URL].